

Minor design project report

Space Partitioning in Last Level Cache

Gautambuddha N Meeshi
2019CS10350

May 2023

1 Introduction

Last level cache in the multicore SOC is a shared cache between multiple cores. The latencies of the LLC is higher than the private caches and it has to process requests and responses of multiple cores. This leads to space and bandwidth contention at LLC.

The space in the LLC has to be efficiently managed to avoid the off-chip accesses which reduce the performance significantly. Traditionally the shared caches used LRU or a pseudo LRU replacement policy to manage the space in the LLC.

LRU replacement policy does not discriminate between cores, may lead to a very unfair space allocations in the extreme cases. Space partitioning in LLC with respect to cores has been proposed by many researchers to improve fairness and performance of the multicore SOC.

The report explains space partitioning methods, space allocation algorithms very briefly. It contains the experimental configuration and experiment results of the implementation. Some suggestions are given for the future work.

2 Space partitioning methods

Here I describe the two ways to implement the space partitioning.

2.1 Space partitioning by indexing the cache block

Alter the LRU replacement policy to incorporate the space partitioning [1].

- Add the core id to each block in shared cache, this will consume $\lceil \log(\text{num_cores}) \rceil$ bits for each cache block. This is used to identify the core that allocated the cache block.

- On a cache miss -
If an empty cache block is present in the set then it is used for allocation. Else, count number of cache blocks belonging to each core in the corresponding set.
 1. If the cache blocks count of the miss causing core is greater than or equal to the allocated number of ways then evict the LRU cache block among the core's cache blocks.
 2. Else, evict the LRU block among the cache blocks of other cores.

2.2 Space partitioning by range allocation

- The cores are mapped to classes. Multiple cores can be mapped to the same class.
- For each class a range is assigned. Multiple ranges can be overlapping.
- The read, write, eviction operations by the requests of the core are restricted to that range only.

3 Space partitioning algorithms

This section explains partitioning algorithms (algorithms that use system stats to decide the space allocation to each core).

3.1 Utility based cache partitioning

Utility based cache partitioning [1] says, provide more ways to the core whose misses will be reduced by providing the ways.

For this [1] tries to estimate the number of misses and number of hits that would happen if the core was allocated x ways, $x \in [1, llc_assoc]$. It proposes an additional hardware called utility monitoring (UMON) circuit to estimate it.

3.1.1 Utility monitoring circuit

1. It maintains an auxiliary tag directory (ATD) of 32 sets for each core.
2. ATD maintains the LRU sequence of addresses accessed by the core. llc_assoc number of addresses are maintained for each set.
3. When an address is accessed, it is searched in the LRU address sequence. If the address is present, the hit counter at that location is incremented. If not present, the miss counter is incremented. The address is moved to the MRU position, other addresses are shifted accordingly. There are $(llc_assoc+1)$ counters for each (LLC, core) pair.

[1] Proposes two algorithms for partitioning as listed below.

Algorithm 1 Greedy UCP

```
for each llc l do
    balance = llc_assoc
    allocation[llc][c] = 0 for each core c
    while balance > 0 do
        for each core c do
            /*get utility of increasing the assoc of core c*/
            alloc = allocations[llc][c]
            u_next = get_util_value(llc, c, alloc, alloc+1)
            winner = core with max value of u_next
            allocations[llc][winner]++
            balance -
        return allocations
    get_util_value(llc, core, alloc, alloc+1):
        U = 0
        /*change in misses for core when the number of blocks assigned to it increases
        from a to b*/
        for i in [a+1,b] do
            U += hit_counter[llc][core][i]
        return U
```

Algorithm 2 Lookahead UCP

```
for each llc l do
  balance = llc_assoc
  allocation[llc][c] = 0 for each core c
  while balance > 0 do
    for each core c do
      /*get utility of increasing the assoc of core c*/
      alloc = allocations[llc][c]
      max_mu[c] = get_max_mu(l,c, alloc, balance)
      blocks_req[c] = min blocks to get max_mu[c] for c
    winner = application with maximum value of max_mu and least blocks
    required
    allocations[llc][winner] += blocks_req[winner]
    balance -= blocks_req[winner]
  return allocations
get_max_mu(llc, core, alloc, balance):
  max_mu = 0
  for i in [1,balance] do
    mu = get_mu_value(llc,core, alloc, alloc+i)
  return max_mu
get_mu_value(llc, core, a, b):
  U = 0
  /*change in misses for core when the number of blocks assigned to it increases
  from a to b*/
  for i in [a+1,b] do
    U += hit_counter[llc][core][i]
  return U/(b-a)
```

3.2 XChange

XChange [2] is a co-optimization based partitioning algorithm. The co-optimization is performed between dynamic voltage and frequency scaling (DVFS) and LLC space partitioning. It is a distributed algorithm, emphasizing on scalability of the algorithm.

3.2.1 XChange with DVFS

1. Central core allocates equal budget to each core and posts the price per resource unit (price of frequency step, price of a cache way).
2. Each core bids to reduce the execution time:
 - (a) Each core buys minimum frequency 800MHz and cache ways with the remaining budget.
 - (b) It reduces the cache ways one by one and uses that budget to buy the frequency step.
 - (c) Finally, it posts the best bid (the combination that gives minimum execution time) distribution between frequency and cache ways to the central core.
3. Central core recalculates the price per resource unit of each resource.

$$p_j = \sum_i b_{ij} / R_j \quad (1)$$

p_j is the price per unit of resource j , b_{ij} is the bid posted by core i for resource j , R_j is the total number of resource units of type j .

4. If the price changes by more than 1% then steps 1, 2, 3 are repeated.
5. Finally, when the prices become stable the resources are distributed proportional to the bids posted by the cores.

$$r_{ij} = b_{ij} / p_j \quad (2)$$

r_{ij} is the number of resource units of resource j to be allocated to core i .

6. As a safeguard a maximum iterations of 30 is set to break the market if convergence does not happen. If the market does not converge, the current bidding and equal share allocation is compared, if anyone of the core prefers the equal share based on execution time then the resources are distributed equally otherwise current biddings are considered.

3.2.2 Estimating the execution time, utility

Cache Utility

1. For each (LLC, core) pair critical path counter (CP_{global}) is maintained to estimate the t_{mem} (memory phase time) under the current allocations.
2. When a request reaches the LLC the CP_{global} is copied into the MSHR, call it CP_{local} .
3. After Δt cycles when the response arrives to LLC, the CP_{global} is updated.

$$CP_{global} = \max(CP_{global}, CP_{local} + \Delta t) \quad (3)$$

4. To estimate the t_{mem} when the core is allocated a different number of ways, it is assumed that memory level parallelism (MLP) is constant.

$$MLP = \frac{N_h * t_h + N_m * t_m}{CP_{global}} \quad (4)$$

5. To predict the $CP_{global}(j)$ (t_{mem} when the core is allocated j ways at an LLC). $N_h(j), N_m(j)$ are estimated using the UMON circuit similar to UCP.

$$CP_{global}(j) = \frac{N_h(j) * t_h + N_m(j) * t_m}{MLP} \quad (5)$$

6. The increase in its execution time can be computed as

$$MU_{cache}(i) = CP_{global}(i - 1) - CP_{global}(i) \quad (6)$$

$MU_{cache}(i)$ is the decrease in t_{mem} when the allocation for the core is increased from $i - 1$ to i .

Power utility

- Let t_{exe}, f_0, E_0, V_0 be the execution time of compute phase, average operating frequency, energy consumption, operating voltage of a CPU in the previous interval respectively.
- Then the length of compute phase $t_{exe}(f)$ under the new frequency f .

$$t_{exe}(f) = t_{exe} * f_0 / f \quad (7)$$

- Power estimate

$$P(f) = \frac{\frac{E_0}{V_0^2} \times V_f^2}{t_{exe}(f) + t_{mem}} \quad (8)$$

- Marginal utility for power.

$$MU_{power}(f + \Delta f) = \frac{t_{exe}(f) - t_{exe}(f + \Delta f)}{P(f + \Delta f) - P(f)} \quad (9)$$

where Δf is the frequency increment of one DVFS step.

3.2.3 XChange without DVFS

In this project we want to perform LLC partitioning only. Without co-optimization each core will bid all its budget for the LLC and the cores will receive the resource proportional to the budget allocated. By default XChange allocates equal budget to each core which leads to equal partitioning of cache among the cores.

However the XChange paper proposes a variable budget allocation method also, to increase the performance at the cost of some fairness.

Heuristic to assign budget/number_of_ways is given in eq(10).

$$B \propto (1 - \frac{t_{min}}{t_{max}}) \quad (10)$$

t_{max}, t_{min} is the sum of t_{exe} and t_{mem} when the core is allocated only one way and all ways respectively.

3.3 Machine learned machines

[3] tries to use the Q-learning algorithm to reduce the off-chip bandwidth access.

Algorithm -

1. Given the associativity and number of cores, generate all possible states and actions.
 $states = \{ \langle S_1, S_2, \dots, S_{num_cores} \rangle : S_i > 0 \text{ and } \sum S_i = assoc \}$
 $actions = \{ \langle a_1, a_2, \dots, a_{num_cores} \rangle : a_i \in \{-1, 0, 1\} \text{ and } \sum a_i = 0 \}$
2. Create a 2D integer vector of size $|states| \times |actions|$ for each LLC as a Qtable.
3. All the messages that are transferred between LLC and Directory controller are considered as off-chip bandwidth access.
4. Every 500,000 cycles the Q table is updated, partitioning is changed. The interval size is chosen much smaller than other algorithms to explore the state space faster.
5. The Q table is updated using the equation (11). Q learning equation

$$Q(llc, s_{t-1}, a_{t-1}) = Q(llc, s_{t-1}, a_{t-1}) + \alpha * (-1 * off_chip_accesses(llc) + \gamma * (\max_a (Q(llc, s_t, a)) - Q(llc, s_{t-1}, a_{t-1}))) \quad (11)$$

6. Next action is selected according to the below equation

$$next_action(llc, q_t) = \begin{cases} actions[randint], & \text{if } random() \leq 0.3 \\ \arg \max_a (Q(llc, q_t, a)), & \text{otherwise} \end{cases}$$

7. If the action selected is invalid (leads to a state where a core has non positive number of ways) then a random action is considered.
8. The MLM partitioning is performed only on 12 ways remaining 4 ways are shared to all the cores (assuming $llc_assoc = 16$).

3.4 Critical phase aware partitioning

This partitioning algorithm [4] classifies the cores into one of the four categories viz., critical, non-critical, bully, squanderer. Critical category is further divided into sensitive and medium based on the IPC.

The classification criteria is given below

Class	IPC	MPKI	HPKI
sensitive	L (> 0.4)	H ($> \text{Eq.12}$)	not VL (≥ 0.5)
medium	M (≤ 0.4)	H ($> \text{Eq.12}$)	not VL (≥ 0.5)
bully	VL (< 0.2)	VH (≥ 10)	VH (≥ 10)
squanderer	$\times$	H ($> \text{Eq.12}$)	VL (< 0.5)
non-critical	otherwise		

Table 1: Thresholds to identify the category

$$th_{MPKI_{LLC,H}} = \max(1, \mu + 1.5 \times std) \quad (12)$$

μ is the average of $MPKI$ of all the cores, std is the standard deviation of the $MPKI$ of all cores.

# critical cores	non critical cores' assoc	critical cores' assoc
1	50	60
2	45	65
3	40	70
0 or more than 3	100	100

Table 2: Percentage of space allocated based on number of cores in each category

core type	#CLOS(es)
bully/non-critical	1
critical	2,3,4
squanderer	5,6

Table 3: Default class assignments

The values of the thresholds have to be set according to the system and the benchmarks to obtain the best result. I have changed the IPC thresholds to suit the gem5 simulator by observation.

Phase change is detected using interval coefficient of variation (ICOV). ICOV is calculated as the ratio of change of current interval IPC with respect to running average of the IPC. $ICOV_{thresh} = 0.2$.

3.4.1 Partitioning algorithm

1. Calculate IPC, MPKI, HPKI of each (llc, core) pair.

Assign the behavior to classes. Change the core to class mapping. Change the range of classes if required.

2. **if** first_interval **then**
 - for each llc $update_clos[llc] = true$
 - else**
 - for** each llc **do**
 - for** each core **do**
 - Compute ICOV
 - get category of core
 - if** $ICOV > ICOV_{thresh}$ & core category change **then**
 - $update_clos[llc] = true$
 - for** each llc **do**
 - if** $update_clos[llc]$ **then**
 - allocateClass(llc)
 - allocateRanges(llc)
-

Find medium cores among the critical cores and restrict the space given to them.

3. **for** each llc **do**
 - for** each critical core **do**
 - if** $IPC[llc][core] > IPC_L$
 - & $LLC_occup[llc][core] > LLC_occup_critical[llc]/2$ **then**
 - behavior[llc][core] = medium
 - if** $critical_cores == 1$ & $medium_cores == 1$ **then**
 - Allocate $assoc/4$ ways to *medium_core*
 - $assoc * 3/4$ is assigned to *non_critical* cores
 - else**
 - Retain at least 1 sensitive_core
 - Restrict the medium cores to $(\#medium_cores)/(\#critical_cores)*\#critical_ways$
-

Find the non critical cores which are over exploiting the space without any usage, restrict them to 2 ways.

4. **for** each llc **do**
 - for** each *non_critical* core
 - $LLC_occup[llc][core] > non_crit_occup/3$
 - & $MPKI[llc][core] < 0.5$ & $HPKI[llc][core] < 0.5$ **do**
 - Allocate the core to a new class
 - Restrict the range of that class to 2 ways
-

Increase the space allocated to critical cores or non critical cores depending on the ipc. This is done once in five intervals.

5. **for** each llc **do**
 - if** IPC sum of critical cores has decreased wrt previous interval **then**
 - Ways shared between critical and non critical cores is given exclusively to the critical cores.
 - else if** IPC sum of non critical cores has decreased wrt previous interval **then**
 - Ways shared between critical and non critical cores is given exclusively to the non critical cores.
-
6. If there are more than 3 critical cores then they are mapped to class 1 (same as non-critical category) and range of class 1 is set to $[0, llc_assoc]$ (whole LLC).
 7. If there are more then 2 squanderer then they are assigned to class 1.

3.4.2 Experimental configuration

The CPA [4] was experimented on Intel Xeon E5-2620 v4 processor. The number of instructions each benchmark executes when run alone for 60 seconds was noted. The experiments with 8 benchmarks was run for those many instructions. The benchmarks which finish early were restarted. The partitioning interval used was 500ms (1B cycles for 2GHz).

We did the experiment on gem5 simulator, thus used the partitioning interval of 5M cycles.

4 Experiments

We use gem5 simulator with ruby memory subsystem simulator [5]. We use SPEC2006 benchmark mixes as shown in the fig 1. We fast forward 1B instructions, warm up the simulator for 50M instructions and record the experiment for 100M instructions.

L1 and L2 cache are private to each core. Therefore, the number of L1 and L2 caches is equal to the number of CPUs.

CPU type	O3	num_llcs	2
ISA	X86	LLC size	4MB each
num_cpus	8	LLC MSHR	32
ROB size	250	LLC assoc	16
Frequency	1GHz	LLC data latency	40
		LLC tag latency	10
L2 size	256KB	L1 D/I size	32KB/32KB
L2 MSHR	16	L1 MSHR	8
L2 assoc	8	L1 assoc	8
L2 data latency	12 cycles	L1 data latency	2
L2 tag latency	4 cycles	L1 tag latency	2

Table 4: Simulator configuration

	memory intensive		LLC intensive		compute intensive		mixed	
	core0	core1	core2	core3	core4	core5	core6	core7
mix1	mcf	mcf	mcf	mcf	mcf	mcf	mcf	mcf
mix2	lbm	lbm	lbm	lbm	lbm	lbm	lbm	lbm
mix3	omnetpp	omnetpp	omnetpp	omnetpp	omnetpp	omnetpp	omnetpp	omnetpp
mix4	gamess	gamess	gamess	gamess	gamess	gamess	gamess	gamess
mix5	soplex	soplex	soplex	soplex	soplex	soplex	soplex	soplex
mix6	milc	milc	milc	milc	milc	milc	milc	milc
mix7	bzip2	bzip2	hmmer	hmmer	leslie3d	leslie3d	wrf	wrf
mix8	bzip2	bzip2	omnetpp	omnetpp	milc	milc	leslie3d	leslie3d
mix9	bzip2	bzip2	leslie3d	leslie3d	bwaves	bwaves	dealll	dealll
mix10	bzip2	bzip2	tonto	tonto	gamess	gamess	leslie3d	leslie3d
mix11	omnetpp	omnetpp	soplex	soplex	tonto	tonto	milc	milc
mix12	omnetpp	omnetpp	wrf	wrf	leslie3d	leslie3d	gamess	gamess
mix13	omnetpp	omnetpp	soplex	soplex	gamess	gamess	hmmer	hmmer
mix14	omnetpp	omnetpp	soplex	soplex	bwaves	bwaves	zeusmp	zeusmp
mix15	omnetpp	omnetpp	soplex	soplex	milc	milc	gamess	gamess
mix16	omnetpp	omnetpp	soplex	soplex	bzip2	bzip2	hmmer	hmmer
mix17	hmmer	hmmer	soplex	soplex	bwaves	bwaves	milc	milc

Figure 1: SPEC2006 mixes

5 Results and observations

5.1 Utility based cache partitioning

- The size of dynamic set sampling was set to 64 ([1] used 32) because the experiments use a larger LLC and more cores, more sets have to be monitored to approximate the core behaviour.
- The UMON circuit fails to capture the LLC accesses if the core never accesses one of those 64 sets among 4096 sets. This can lead to a bad partitioning.

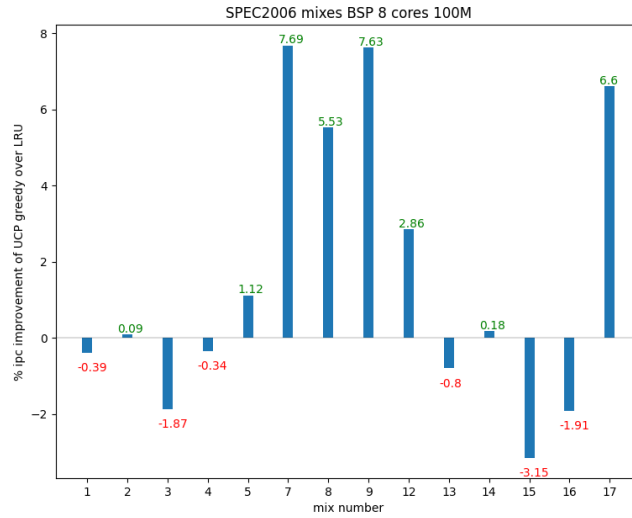


Figure 2: average IPC improvement, UCP greedy vs LRU

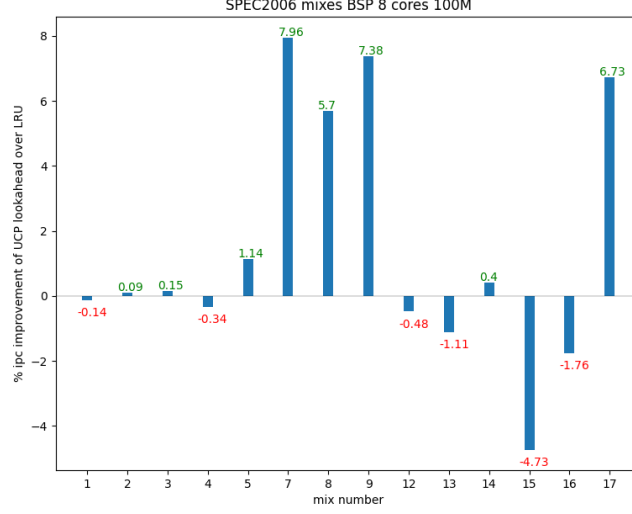


Figure 3: average IPC improvement, UCP lookahead vs LRU

5.2 XChange

- This is a simple partitioning method which considers a lot of parameters ($\#misses$, $\#hits$, $miss_latency$, $hit_latency$, t_{cpu} , t_{mem}). These many parameters make the algorithm very sensitive to the core behaviours. These stats are collected for all the requests for the current allocation. UMON is used to approximate these numbers for different allocations.
- Usage of more statistics can be reason for it to outperform other partitioning algorithms in the best case.
- In gem5 the range of each parameter varies a lot. For example, number of hits can vary from 0 to 3500 for a core depending on the benchmark. Hit latency can vary from 90 to 500 cycles. This much variation can lead to very skewed partitioning.

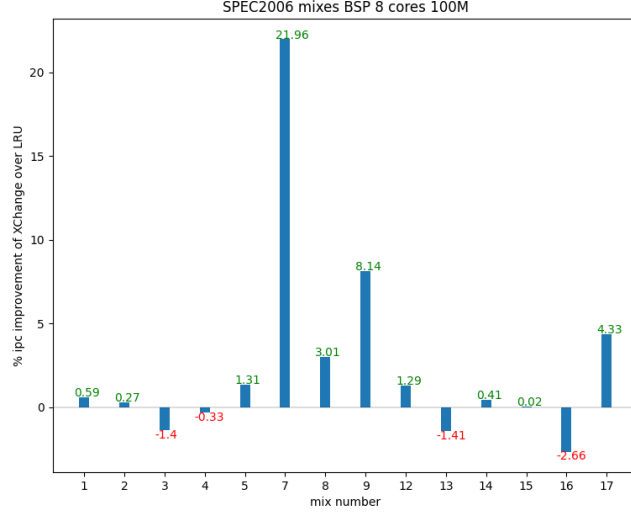


Figure 4: average IPC improvement, XChange vs LRU

5.3 Machine learned machines

- The performance is not much different than the LRU because the partitioning algorithm can change only 1 way per core in an interval. Since the model is not pre-trained, it needs to take many random steps to obtain a meaningful Q-table. [3] suggests to do the partitioning only on 12 ways and keep the 4 ways shared to all the cores.
- It might fail to react to a new behavior.
- This algorithm was run for 250M instructions, with partitioning interval of 500K cycles on (4 core, 1 LLC) and (8 core, 2 LLC) configurations.
- The algorithm has very poor scalability. The model size increases exponentially with increase in the number of cores and associativity of the LLC. A large model implies it requires a lot of data to be trained, more inference time, more memory required for the model.

cores	assoc	# states	# actions
4	16	455	19
8	16	6435	1107
16	24	490314	5196627

Table 5: Table showing poor scalability of MLM

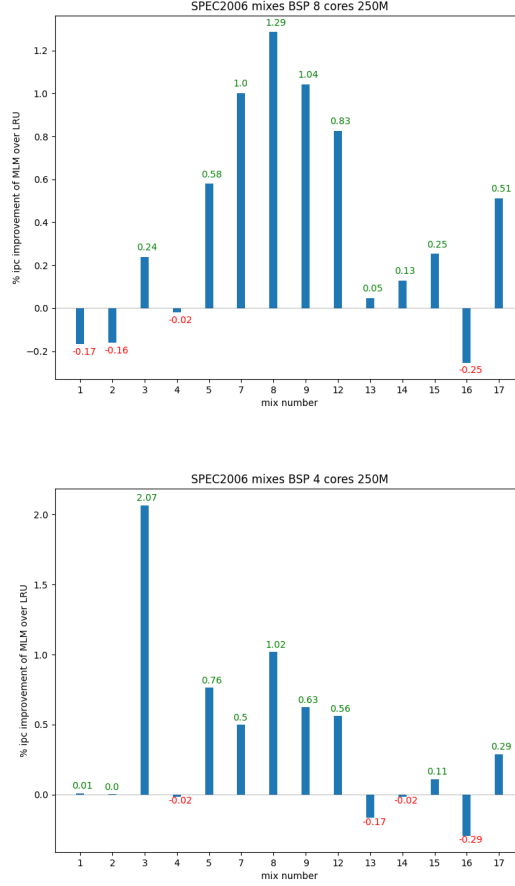


Figure 5: average IPC improvement (8 cores and 4 cores), MLM vs LRU

5.4 Critical phase aware partitioning

- It is a very complex algorithm, has 5 steps. It requires a lot of book keeping for implementing the algorithm.
- It employs a different partitioning method than previous algorithms.
- More emphasis is put on not degrading the performance wrt default configuration. If there are more than 3 critical cores then both critical and non critical cores are mapped to use the whole LLC (they share the whole LLC).

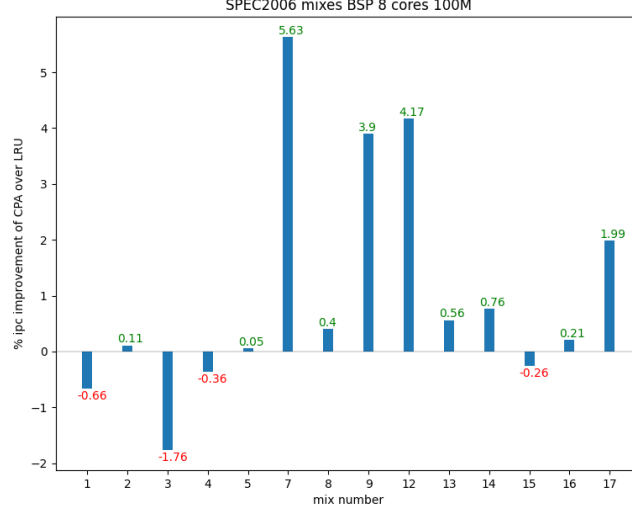


Figure 6: average IPC improvement, CPA vs LRU

6 Other observations

1. It is possible in gem5 that the LLC hit latency of requests of a particular core is more than the LLC miss latency.

This happens when a core accesses the LLC very few times and other cores are accessing the core very frequently. There are three queues going into the LLC viz., response queue, unblock queue, request queue. There queues are processed in the order specified i.e., the LLC will not read the request queue until the response, unblock queue are empty. If the core has very few requests its request messages can be stuck at the request queue for very long time.

2. The benchmarks, simulator experimented is such that the IPC of no core was greater than 1.
3. More performance can be extracted out of the above algorithms by parameter tuning according to the simulator.
4. The above experiments indicate that a simple algorithm that considers more statistics properly performs better.
5. Most of the algorithms perform good when there is proper mix (different type) of benchmarks. They perform poorly for homogeneous benchmarks.

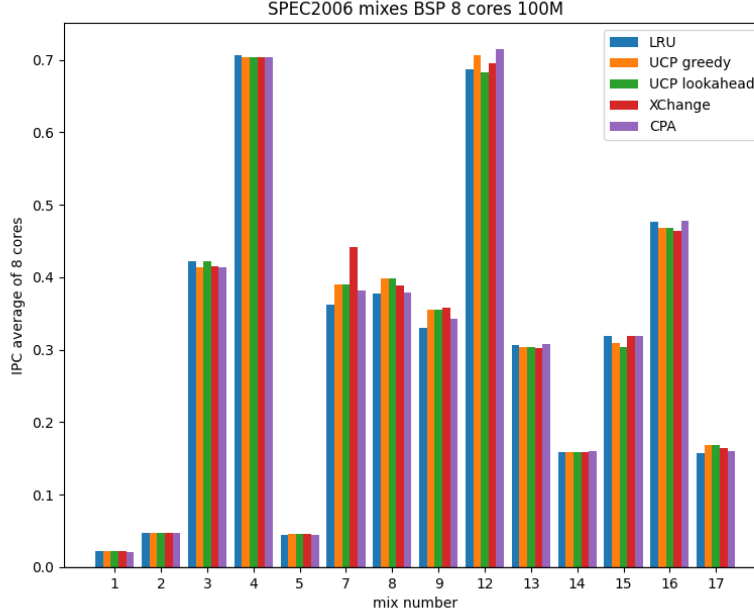


Figure 7: IPC average of LRU, UCP greedy, UCP lookahead, XChange, CPA respectively

7 Contribution

1. Improved the blocking mechanism in the L2 cache and LLC over my previous implementation. In the previous implementation the number of cycles the cache blocked was governed by the message type.
Now, number of cycles the cache blocks is decided by state transition and actions that are performed. This blocking implementation is very accurate.
2. Changed the configuration of the simulator by studying the source files.
3. Implemented way wise space partitioning as suggested in [1], by indexing the cache blocks and altering the replacement policy.
4. Implemented another way wise space partitioning method as suggested in [4], by restricting the search space to the range of the class. In this method the ways allocated are contiguous and multiple classes can share the overlapping ways.
5. Implemented UCP, XChange, MLM, CPA space partitioning algorithms with some parameter tuning.

8 Future work

- The stats that were recorded in the previous partitioning intervals should be interpolated to the current interval, so that the partitioning that will be performed in the current interval is according to the current needs. The interpolation can be done using polynomial interpolation, RNN etc.
- Partitioning should be performed at a finer level. All the above partitioning algorithms enforce the partition for the complete LLC, cores might be having different behaviors at finer level.

We can do the partitioning at finer level as follows. We can divide the LLC into four or more logical parts using set number. Allocate the ways to cores differently in each of the smaller parts of the LLC.

- If we want to use ML techniques then they should be pre-trained.

References

- [1] M. K. Qureshi and Y. N. Patt, "Utility-Based Cache Partitioning: A Low-Overhead, High-Performance, Runtime Mechanism to Partition Shared Caches," 2006 39th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO'06), Orlando, FL, USA, 2006, pp. 423-432, doi: 10.1109/MICRO.2006.49.
- [2] X. Wang and J. F. Martínez, "XChange: A market-based approach to scalable dynamic multi-resource allocation in multicore architectures," 2015 IEEE 21st International Symposium on High Performance Computer Architecture (HPCA), Burlingame, CA, USA, 2015, pp. 113-125, doi: 10.1109/HPCA.2015.7056026.
- [3] R. Jain, P. R. Panda and S. Subramoney, "Machine Learned Machines: Adaptive co-optimization of caches, cores, and On-chip Network," 2016 Design, Automation & Test in Europe Conference & Exhibition (DATE), Dresden, Germany, 2016, pp. 253-256.
- [4] L. Pons, J. Sahuquillo, V. Selfa, S. Petit and J. Pons, "Phase-Aware Cache Partitioning to Target Both Turnaround Time and System Performance," in IEEE Transactions on Parallel and Distributed Systems, vol. 31, no. 11, pp. 2556-2568, 1 Nov. 2020, doi: 10.1109/TPDS.2020.2996031.
- [5] Jason Lowe-Power, Abdul Mutaal Ahmad, Ayaz Akram, Mohammad Alian, Rico Amslinger, Matteo Andreozzi, et al. 2021. The gem5 Simulator: Version 20.0+. ArXiv Preprint ArXiv:2007.03152, 2021.